

AIM

MD.030 DESIGN STANDARDS

<Company Long Name>

<Subject>

Author: <Author>

Creation Date: April 23, 1999

Last Updated: August 15, 2025

Document Ref: <Document Reference Number>

Version: DRAFT 1A

Approvals:

<Approver 1>

<Approver 2>

ORACLE®

Copy Number _____

Document Control

Change Record

Date	Author	Version	Change Reference
23-Apr-99	<Author>	Draft 1a	No Previous Document

Reviewers

Name	Position

Distribution

Copy No.	Name	Location
1	Library Master	Project Library
2		Project Manager
3		
4		

Note To Holders:

If you receive an electronic copy of this document and print it out, please write your name on the equivalent of the cover page, for document control purposes.

If you receive a hard copy of this document, please write your name on the front cover, for document control purposes.

Contents

Document Control	ii
Introduction	1
Purpose	1
Background.....	1
Scope and Application	1
Related Documents.....	1
Overview	2
Basic Business Needs.....	2
Audience.....	2
Design and Development Goals	2
Century Date Compliance	4
Definitions	5
Additional Documents.....	6
Design Document Components	7
Functional Design.....	7
Technical Design.....	9
Final Design.....	11
Topical Essay Standards	13
Form Cosmetic Standards.....	14
Exceptions to Oracle Standards	14
Extensions to Oracle Standards	14
Report Cosmetic Standards	15
General Report Guidelines	15
Boilerplate and Text Standards.....	15
Report Title Standards	16
Cover Page.....	17
Label Standards.....	17
Page Headers.....	17
Page Footers (Legends)	18
Page Parameters.....	18
Messages	18
Column Ordering	18
Formatting Numbers.....	18
Monetary Amounts	19
Non-Monetary Amounts	19
Totals	19

Database Design Standards	20
Database Design Tools	20
Database Design Components	20
Database Naming Standards.....	21
Interface Standards (Messages).....	24
Message Dictionary	24
Message Numbers	24
Message Naming Standards.....	24
Message Writing Standards	25
Naming Standards.....	28
Open and Closed Issues for this Deliverable	30
Open Issues.....	30
Closed Issues	30

Introduction

Purpose

This document describes the standards that <Company Long Name> will follow on the <Project Name> when designing customizations to the Oracle Applications™.

Background

The information in this document has been defined as the result of discussions between project team representatives, <Company Short Name> technical staff, and <Consulting Services Provider> consultants.

Scope and Application

The standards in this document cover the Solution Design phase and will primarily affect tasks in the Module Design and Build process of AIM:

Related Documents

1. Application Extension Strategy for <Project Name>.
2. *Oracle Applications User Interface Standards*

Overview

This document describes the design standards adopted by <Company Long Name> for customizations and extensions to Oracle Applications.

Oracle Applications are developed in adherence to strict design and development standards. These standards help insure high quality, portability across multiple platforms, consistent look and feel, ease of maintenance, and compatibility with future versions of Oracle tools and Application Object Library.

Customizations, modifications, or extensions to Oracle Applications must also follow strict standards for many of the same reasons. They must also take into consideration factors such as ease of upgrades, preservation of core functionality, maintainability, and the design review process.

Basic Business Needs

The application customization standards set forth in this document are designed with the following in mind:

- Minimize the impact on core application functionality.
- Permit easy upgrades to future releases of Oracle Applications.
- Allow customizations to be easily reinstalled.
- Provide complete and easily understood documentation.
- To assist customizations in meeting user requirements.

Audience

This document is intended for the following individual:

- <Company Short Name> technical staff members
- Outside consultants
- Reviewers of design documents

Design and Development Goals

Customizations are Protected from Upgrades

All custom code is stored in a separate directory from production code, even if it is a modification of a standard program. Likewise, custom tables and lookup codes are created in a separate Oracle user account. You define custom menus and responsibilities to access the custom modules. Upgrades to the standard product affect only the standard production code and database objects.

Customizations are Thoroughly Tested

Unit and link test plans are prepared during the design phase of a customization. This allows the design reviewer to confirm that the testing will validate all required functionality before coding is even started. It is also a useful cross-check for the developer during the coding phase.

When possible, someone other than the developer will test the customization. Users are required to retest the customization after installation before the customization is considered complete.

Customizations can be Easily Re-installed

Installation scripts and detailed instructions allow customizations to be easily reinstalled in a new environment or at another site. This is especially important when customizations are part of a global solution in a multiple site implementation.

Customizations are Non-Intrusive

Customizations to Oracle Applications should never change core functionality. The standard functions built into the software will always remain unchanged. This policy preserves our support agreement with Oracle Worldwide Support and provides the highest level of maintainability.

Customizations provide additional functionality to extend the base product. In some cases a customization may need to alter the results of an existing process such as MRP planning, PO generation, or forecast loading. In these cases, you should design the customization as an add-on process that runs before or after the standard process instead of attempting to rewrite the standard process.

If a standard form or report must be modified, it is copied to the custom application directory, modified, then registered under the custom application. The menu must then be modified to call the new version of the form or report. The original program remains in its original location.

Designs Follow Oracle Documentation Standards

Design documents for customizations to Oracle Applications should follow the style conventions of the standard Oracle documentation. Functional design documents include a topical essay, report descriptions, and form descriptions. These are presented in the same format as the application reference manuals and become the standard user documentation for the customization.

To ensure consistency, all designers will use the Application Extensions Functional Design and Application Extensions Technical Design templates included in AIM to create all documents.

Code Adheres to Oracle Applications Development Standards

Coding for customizations to Oracle Applications follows the standards outlined in *The Application Object Library Reference Manual* with some qualified exceptions. Standards that should always be followed are:

- Application Object Library development standards.
- Form and report cosmetic standards.

- Comment standards.

Exceptions include:

- Porting requirements for platforms that <Company Short Name> will never port to may be waived if it saves significant development effort.
- Internal library routines not supported by Oracle should not be used.
- Regression test scripts may not be necessary.
- Support of multiple languages may not be necessary.

In general, any time you deviate from the documented standards, make sure you have a reason. If the deviation is in source code, indicate the differences and reasons with comments.

Scheduled Sign-Off Reviews

Reviews are scheduled at various points in the design and development process. Reviews will assist in making certain that the design of the customization properly addresses the needs of the user and includes all the required functionality before coding is started. A formal sign-off is used to document the users' agreement and approval that the design accurately satisfies the requirements.

After the initial requirements definition, the first design review occurs after the topical essay is written. The goal of the topical essay is to summarize the designer's understanding of the business needs and present the approach that will satisfy those needs. This review is fairly informal and will assist in being certain that the designer is on track before commencing with additional functional design.

The second review occurs after the functional design is complete. The functional design includes the topical essay, test plans, open/closed issues, plus detailed descriptions of each form, report, and process included in the customization. Form, report, and process descriptions are from the user perspective and do not include table names, program logic, or other technical details that users may not understand.

The final design review is performed when the technical design is complete. The technical design includes all of the functional design plus technical descriptions of each module, database design, and integration issues. It may also contain refinements to the functional design sections.

A final sign-off is performed after the customization is installed and tested by users. This is the final acknowledgment that the customization satisfies all the requirements and there are no known bugs.

Century Date Compliance

In the past, two character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four character date encoding. As the year 2000 approached, it became evident that a full four character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used.

Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

When designing and building application extensions, it is essential that all dates be entered, stored, and processed using the full four digit year for compliance with Century Date standards. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Definitions

Customization

A custom developed module that provides additional functionality to an Oracle Application. Customizations include new modules that provide features not provided by standard applications and also modifications and extensions to existing applications.

Modification

A change to an existing application module that changes core functionality. Modifications are discouraged.

Extension

A customization that adds to the functionality of an existing application or module through additional forms, form zones, reports, or a supplemental process.

Module

A single form, report, user exit, stored procedure, database trigger, or concurrent program. A single customization may include multiple modules.

Enhancement

New features or changes to existing applications that will become part of the base product in future releases and will be supported by Oracle. Enhancements are designed and implemented by the Application Development organization of Oracle.

Interface

An extension that establishes an automated linkage between the Oracle Applications System and legacy or 3rd Party Systems. The linkage can consist of an outbound transfer of data from Oracle Applications to an external system, or an inbound transfer of data from an external system to Oracle Applications.

User

A person who will use the customization. Users also establish the requirements that a customization must satisfy.

Designer

The person who interprets the user requirements and designs an approach for meeting those requirements. Designers may be functional, technical, or both.

Developer

A person who codes customizations.

Project Manager

The person who is responsible for the project schedule and cost management. The project manager makes the decision to build customizations based on cost/benefit analysis.

Additional Documents

The following Oracle documents provide additional information about application development standards:

- ☐ *Oracle Application Object Library Reference Manual*
- ☐ *Oracle Applications Installation Manual*
- ☐ *Oracle Applications User Interface Standards*

Design Document Components

The goal of the design document is to clearly communicate all of the features of the customization. It should contain all the information necessary to educate someone who has no prior knowledge of the requirements or business environment.

There are four stages of the design document and each is written for a different audience. Each stage incorporates the components of the prior stages.

Design Stage	Audience	New Components
Functional Design	Users, Technical designer	Topical Essay Form and Report Descriptions Concurrent Program Descriptions Technical Overview Open/Closed Issues
Technical Design	Developers	Form Logic Concurrent Program Logic Integration Issues Database Design Installation Requirements
Final Design	<Company Short Name> MIS Staff	Implementation Notes

The final design stage is completed after the customization is complete. The complete design document becomes the on-site technical reference manual for the customization and should be updated whenever additional features are added.

Functional Design

The goal of the functional design is to describe all of the features of the customization as they will appear to the user. It also serves as the user reference manual for the customization. At a minimum, the functional design should include form and report descriptions presented in the same format as the standard Oracle Application reference manuals.

When writing a functional design, keep in mind the user who will be reading it. Does it adequately explain the fields? Does it help them understand how to use the form?

Topical Essay

The goal of the topical essay is to summarize the requirements and present the features of the solution. It should also explain how the customization is incorporated into the normal business operations.

A topical essay includes the following sections:

Introduction

Briefly describes what the customization is and lists the components of the customization.

Basic Business Needs

Lists the business needs that the customization must satisfy. These should be at a higher level than functionality.

Major Features	The major features of the customization and the benefits they provide. The description of each feature should be fairly brief. Additional detail can be added in the User Procedures section or in the form and report descriptions of the Functional design.
Definitions	Definitions of unique terms referenced in the design document.
User Procedures	The steps the user will follow to use the customization.
Examples	(Optional) One or more examples that illustrate the features.
Diagrams	(Optional) Entity relationship diagrams and data-flow diagrams should be included if new database entities are being added and there are major processes that interact heavily. Oracle Designer should be used if it is available.
Assumptions	Any assumptions regarding the business process or environment that the design is based upon.

Additional sections may be added when it makes sense to do so (typically between Definitions and Assumptions). Diagrams and illustrations may be added where appropriate. Remember that the topical essay will become the primary user documentation. (See the topical essays included in the Application Reference manuals for good examples.)

Form and Report Description

Form descriptions include the layout of each zone of the form, and descriptions of each zone and field. Field descriptions include List of Values (LOV) and relevant validation logic (expressed in user terms).

Report descriptions include the required input parameters, sample report output, and descriptions of each column and data element on the report. The derivation of calculated values should be included.

Concurrent Program Description

Concurrent Program Descriptions include the required input parameters, sample log output, and a description of the changes to business data.

Technical Overview

Describe the technical approach planned to implement the customization. Do not include detailed logic that is specific to a module — this section should describe the overall technical flow and how the individual modules will work together.

Open/Closed Issues

As issues are identified throughout the design and development process, they must be documented here. Issues are divided into two sections:

Open Issues

All unresolved issues are assigned a number and included here. Possible resolutions may be listed.

Closed Issues

When an issue is resolved, it is moved from Open Issues into Closed issues (the issue number is not changed). An explanation of the resolution must also be included.

Technical Design

The technical design document is the set of instructions given to a developer to code the customization modules. It must include all calculations and conditional logic required in the code.

Technical Overview

This is carried forward from the Functional Design and expanded for the Technical Design. Describe the technical approach planned to implement the customization. Do not include detailed logic that is specific to a module – this section should describe the overall technical flow and how the individual modules will work together.

Form Logic

A form logic section includes the following:

Overview

Technical overview of the form. It should highlight unique features or special navigation requirements.

Navigation Logic

Describes the sequence of events that occur as a user enters or views data in a form. It can include pseudo-code conditional logic where appropriate. Separate logic should be included for inquiry and update modes.

Table and View Usage

List of tables accessed with the type of access indicated.

Validation Logic

Description of the required validation of each field.

List of Values(LOV) Logic

Description of the source of values in List of Values (LOV) lists.

Default Values

Default values for each field (when a new record is created).

Concurrent Program Logic

This section is used for reports and other concurrent programs including interfaces. It includes:

Overview

Technical overview of the program.

Calling Arguments	Description of each argument passed to the program, its data type, and valid values.
Log Output	Sample output of concurrent manager log (this is distinct from report output that is documented in the Report Description).
Table and View Usage	List of tables accessed with the type of access indicated.
Program Logic	Pseudo-code of program logic. Recommended pseudo code style is based on PL/SQL.
SQL Statements	Specific SQL statements used to select, insert, update, and delete data in the program.
Default Data Sources	For a report, this lists the source of values included on the report. For programs that update tables or write interface files, this lists the source of each data element. This can include conditional logic for derived data.
Validation Logic	Specific validations. It also includes error and warning conditions.
Incompatibility	List of other programs that are incompatible. This information is used when the program is registered in Application Object Library.
Performance Considerations	Descriptions of factors that may affect performance.
Other Considerations	Any other considerations plus description of a restart strategy and crash recovery techniques.

Integration Issues

This section describes any issues that affect other products or other custom modules. It includes:

Changes Required	Any changes required in other custom programs to support this customization. Usually changes or extensions to standard product modules is covered elsewhere in the design document. This topic covers changes in other customizations that may have already been designed.
Shared Components	List of tables, views, and sequences that are owned by other products that this customization uses.
Alert Conditions	Any conditions that are candidates for Oracle Alert messages.
Incompatibilities	List of incompatibilities with other product modules.

Performance Issues

Performance issues that are influenced by other products. This information can be used to determine optimal tablespace location.

Database Design

This section describes changes to the database definition or contents required to support the customization. It includes:

Desired Table Changes

Description of changes to existing tables. Customizations should only include changes to other custom tables and not standard product tables.

New/Updated Seed Data

New rows that must be added to lookup tables. Customizations should have their own shared lookup table (e.g., CST_LOOKUPS).

Descriptive Flexfields

Definition of descriptive flexfield segments required to support the customization.

Value Sets

Value sets required for descriptive flexfields.

Grants/Synonyms

Defines owner of data base objects and lists grantees.

Archiving

Description of what data may need to be archived and an archiving strategy.

Database Diagram

Diagram of actual tables and relationships (a refinement of the E/R diagram included in Functional design).

Tables, Indexes, Sequences

Detailed description of each new database object. This can be the actual SQL statements required to create the objects. This information should be defined in Oracle Designer. CASE reports can be attached or imported into the design document.

Installation Requirements

This section describes the steps required to install the customization. This is not the installation instructions, but is used as a guide to writing installation scripts.

Final Design

The final design includes any updates to the design document made during development, completed test plans, copies of sign-off sheets, scope control documents, and an implementation component.

The implementation component describes how the customization was implemented. It includes:

Design Summary

Description of the design process followed for this customization. It also includes a description of sign-off procedures and who approved each design.

Coding Summary

Description of the coding phase. It also names the developers assigned to each module.

Testing Summary

Description of how the customizations were tested and who performed the testing.

Installation

Description of how the modules are installed, location of files, custom menus, etc.

Topical Essay Standards

Form Cosmetic Standards

Form cosmetic standards are governed by the standards published by Oracle in the following documents:

☐ *Oracle Application Object Library Reference Manual*

☐ *Oracle Applications User Interface Standards*

Exceptions to Oracle Standards

Follow all standards defined in the Oracle standards manuals except for the following exceptions:

- Extra horizontal space in form boilerplate is not required since we do not need to translate forms to other languages.

Extensions to Oracle Standards

You must abide by the additional standards below that are not described in the Oracle standards manuals.

<Standard 1>

<Explanation>

<Standard 2>

<Explanation>

Report Cosmetic Standards

Every report you design can help or hinder the productivity of your users. A well-constructed report has a clear purpose, organizes information strategically, and displays it in an attractive format. Each report should provide all the information your users need for a specific task.

Good report cosmetic standards do not necessarily ensure reports are accurate or thorough. They do improve a report's legibility and ease of use. Following cosmetic standards also ensures consistency among reports that helps a user understand a particular report.

General Report Guidelines

Show Only Fields Related to Our Report's Business Purpose

Make your report easier to read and use by eliminating unnecessary fields that do not relate to your report's business purpose.

Minimize the Number of Fields in a Report

The more fields a report displays, the more difficult it is to read and use effectively for any one purpose

Try to Design a Separate Report to Satisfy Each Business Purpose

If you try to make a single report serve many business purposes, you are almost certain to write a cumbersome report containing too many fields.

Boilerplate and Text Standards

Use Mixed Case Throughout Your Report

Use mixed case wherever you can. Use all capitals only for generally accepted industry acronyms (such as LIFO). Mixed case reports are easier to read than all capitals and are consistent with the standard Oracle Applications reports and forms.

Avoid Abbreviations

Avoid abbreviations when possible. They are hard to read and understand. When you cannot avoid abbreviating, use the same abbreviation for that term throughout the report.

Use Consistent Field Labels Across Reports and Forms

Ensure that a particular field has an identical label on each report in which it appears. When you use the same label in each report that shows a field, your users can easily decide whether two reports show comparable or similar pieces of information.

You should also use matching terminology or field labels between reports and forms. This helps your users understand whether the data they enter in a form prints in a

particular report and where to enter that data. If room is available, you may add additional descriptive text to the report label.

Left Align Alphanumeric Values

Left align the values in an alphanumeric column.

Right Align Numeric Values

Always right align numeric values.

Use Consistent Field Widths Across Reports

When a field appears on several reports, ensure it has the same width on each such form.

Many forms contain scrollable fields that display fewer characters than they actually hold. In these cases, make your report match the actual field width of a form rather than the display width.

Exception: You do not always need to print the entire width of long names, descriptions, or comment fields (which can be up to 240 characters). If the business purpose for a particular report does not require you to print long field in full and your users can nevertheless benefit from a shortened field, then truncate the field rather than omitting it.

Express Ranges Using *From: To:* Format

When you show a starting point and an ending point, for example a start date and an end date, use the following format with stacked left labels:

```
Range Name From:
                To:
```

Example:

```
Vendor Numbers From: 0
                  To: 99999
```

When you do not label the form and the fields, such as in a report subtitle, use the following format:

```
Vendor Numbers from 0 to 99999
```

Report Title Standards

Title a Report Based Upon its Primary Purpose

Briefly state a report's primary purpose in its title. Avoid unnecessary words.

Right: Prepayments Status Report

Wrong: The Status of Customer Prepayments Report

Use a Subtitle to Distinguish Between Related Reports for Differing Audiences

Several reports may show identical or similar information, but they each present report information differently for a separate audiences. Indicate the intended audience in the report subtitle.

Examples:

Resource Account Distribution Report
Account Summary

Resource Account Distribution Report
Distribution Detail

Cover Page

Optionally Include a Cover Page to Show Report Parameters and Report Information

If the report supports many different options, include a cover page that lists all parameters.

Label Standards

The following standards apply to column headings and other boilerplate field labels:

- Use a dashed line to indicate the width of a field.
- Left align alphanumeric top labels; right align numeric top labels.
- Use a colon and two blank spaces after a left label.
- Align stacked left labels at the colon.
- Precede and follow each stacked set of fields with a blank line.
- Label a field using its form field prompt.
- Use spanning headers to fit long top labels.
- Use a minimum of 30 characters for wrapped text columns.

Page Headers

The following standards apply to the repeated headings at the top of each page:

- Show report name, report coverage information, date, and page number in a page header.
- Skip two blank lines after a page header.
- Identify an organization or product source in a page header.
- Include the report title and subtitle in a page header.
- Show a time span, report currency, and scaling factor in a report subtitle.
- Label a confidential report below the title and subtitle.

- Include a timestamp and page number in the top right corner of a page header.
- Show other important report parameters in a page header.

Page Footers (Legends)

The following standards apply to the repeated footers at the bottom of each page:

- Provide a page footer legend if your report contains symbols or abbreviations.
- Legend a column to avoid a wrapped report.
- Use pipe characters to label a field that uses a legend.
- Make single-column legend to standard.

Page Parameters

The following standards apply to information specific to a page when a report groups information onto separate pages:

- Display page parameters in a header line.
- Use page parameters to reduce the number of nested detail levels.

Messages

The following standards apply to special messages:

- Use *****No Data Found***** to indicate a report ran but found no data.
- Use *****End of Report***** to indicate the end of a report.

Column Ordering

The following standards apply to the order in which columns are arranged on a report:

- Order columns logically and consistently across reports.
- Put user key columns on the left side of a report.
- Put most important columns at the far left or far right of a report.

Formatting Numbers

The following standards apply to numeric values:

- Use leading zeros for values less than one (1).
- Align decimal numbers at the decimal point.
- Print negative numbers using a preceding floating minus sign.

Monetary Amounts

The following standards apply to numbers that represent monetary amounts:

- Indicate a currency on each report that shows monetary amounts.
- Do not show currency symbols in a report.
- Display the currency code in the header of a single-currency report.
- Show the scaling factor (if needed) in the header of a single-currency report.
- Show a currency code next to each value or column of values in a multiple currency report.

Non-Monetary Amounts

The following standards apply to numbers that represent amounts that are not monetary:

- Indicate the unit of measure for each amount unless it is obvious.

Totals

The following standards apply to computed totals:

- Use a descriptive left label to preface a column total.
- Distinguish between a subtotal left label and a total left label.
- Indicate and distinguish between a subtotal and a total.

Database Design Standards

Database Design Tools

Oracle Designer

Our primary tool for designing and documenting database extensions will be Oracle Designer. We will install the Oracle Applications CASE database as the base model and add custom applications in the Oracle Designer database that correspond to each application that requires database extensions.

AIM Template

Use the Database Extensions Design (MD.060) template to organize and create the text portions of your design.

Database Design Components

As part of the design process for customizations, we will create a single Database Extensions Design that documents all the database extensions required to implement all the planned customizations. This document includes the following components:

- Overview
- Data Model
- Logical Database Design
- Index Design
- Physical Database Design
- Flexfield Design

Each of these components contains specific details as described below.

Overview

The overview provides a high-level description of the database extensions. For example, it should list the specific applications that are impacted. If we are using a specific technique heavily, such as views that transform data, the overview should describe the technique and the benefits it provides.

Data Model

The data model provides a graphical representation of new business objects and entities in the form of an Entity/Relationship (E/R) diagram. Include existing application entities and the relationships between standard and new entities.

Use the Entity/Relationship Diagrammer in Oracle Designer to model the new entities and the relationships with existing application entities.

Logical Database Design

The logical database design defines the actual tables, views, and other database objects required to implement the data model and support the required functionality. This component includes:

- Database Diagram(s)
- Table List
- View List
- Table and View Definitions
- Required Grants and Synonyms

Use the Oracle Designer Database Diagrammer to create the database diagrams. A database diagram differs from an E/R diagram in that it depicts actual tables and foreign key relationships instead of business entities.

Index Design

The index design defines what columns of each table to index and the type of index required. Your decisions will be based on the specific data usage by custom modules and the anticipated data volume. Work closely with the module designers to understand the specific requirements.

Physical Database Design

The physical database design assigns the specific Oracle tablespace and sizing parameters for each table and index. You must consider the growth and usage properties of the data. For large tables with high query loads, you should consider disk striping strategies.

Flexfield Design

The Flexfield Design captures common flexfield definitions across business areas. Some flexfield requirements will be derived from conversion data mapping while others are defined by module designers to support special functionality. This consolidated design prevents two designers from choosing the same flexfield segment name for different uses.

Database Naming Standards

Database object names have the following general format:

<Prefix>_<Description>_<Suffix>

Database object names are limited to 30 characters. Since some database object names are derived from a base table name (such as sequences and indexes), limit table names to 27 characters to allow for a suffix.

Prefix

Use the Application Short Name for the custom application as the table prefix. All <Company Short Name> custom applications will be prefixed with <Application Prefix (UPPERCASE)> concatenated with the two or three-letter abbreviation of the

standard application that is the subject of the customizations. For example, the custom application for extensions to Purchasing is <Application Prefix (UPPERCASE)>PO.

Column names do not require a prefix

Description

Use multiple words separated with underscores to describe the database object. Use names that a user can recognize. Use plural names for all tables and views. Use singular names for columns.

Standard Abbreviations

Use the following standard abbreviations for common name components:

- AVG Average
- CR Credit
- DR Debit
- FIFO First in, first out
- LIFO Last in, first out
- ID System-generated unique ID
- JE Journal entry
- LEN Length
- MAX Maximum
- MIN Minimum
- MISC Miscellaneous
- MTD Month to date
- NUM System generated number
- PTD Period to date
- QTD Quarter to date
- SEQ System-generated sequence number
- SHIP Shipping
- SPEC Specification
- STD Standard
- TXN Transaction
- UOM Unit of measure
- VAT Value-added tax
- YTD Year to date

Try not to create unique abbreviations for your table names. Exceptions to this guideline are: The abbreviation is a standard industry acronym such as MRP (Material Requirements Planning). You cannot construct a meaningful name with the allowed 27 characters. If you do create your own abbreviations, be consistent and use the same abbreviation in all object names.



Attention: Submit new abbreviations to <Standards Manager> for inclusion in the sanctioned list.

Suffix

Use the following standard suffixes for database objects that are not tables:

- **_V** View
- **_Sn** Sequence (where *n* is a number)
- **_Un** Unique index (where *n* is a number)
- **_Nn** Non-unique index (where *n* is a number)
- **_PK** PL/SQL Package
- **_TRG** Database trigger

Always use the base table name in conjunction with suffixes for sequences and indexes. The description component of a view name should reflect the information that the view selects.



Attention: If you are using a view as a direct substitute for a standard table, you do not need the **_V** suffix.

Column Suffixes

Use the following standard suffixes for database column names:

- **_ID** Surrogate keys
- **_SEQ** Sequence numbers
- **_FLAG** Yes/No QuickCode columns
- **_CODE** Other QuickCode columns
- **_NAME** Spelled-out, user-visible columns
- **_NUM** User-supplied primary keys that are numbers
- **_PERCENT** Percentages, which are normally in the range 0 to 100



For a summary of naming standards see the section entitled Naming Standards.

Interface Standards (Messages)

Interface standards dictate how modules provide information, warning, and error messages to the user. This is not to be confused with user interface standards for forms. All types of modules need to follow a common set of standards for communicating status messages.

Message Dictionary

Use message dictionary for all messages displayed from Forms. All custom messages must be registered with the custom applications we create for customizations.

Message Numbers

Format

All messages will be formatted as follows:

PFX-nnnnn

PFX represents the prefix, which corresponds the application short name for each of our custom applications and nnnnn is a five-digit number.

Number Ranges

Use the following number ranges for the types of modules indicated:

Oracle Forms <range start> - <range end>

Oracle Reports <range start> - <range end>

PL/SQL <range start> - <range end>

SQL <range start> - <range end>



Attention: Oracle Applications use the prefix APP and numbers in the range 00000 through 40000.

Message Naming Standards

Use Descriptive Words and Underscores

When using Message Dictionary, messages names must be no more than 30 characters. The user never sees the message name, but use words that are meaningful to other developers. Use underscores to separate words.

Right: CBOM_CST_DATE_OUT_OF_RANGE

Wrong: CBOM CST DATE OUT OF RANGE

Use All Capitals in Each Message Name

Include only capital letters and underscores in a message name. Although Message Dictionary is not case-sensitive, your message names are easier to query and type accurately when you follow this standard.

Use the Custom Application Short Name as the First Word

Include the custom application short name at the beginning of the message name. For example, the custom application we are using for extensions to GL is <Application Prefix (UPPERCASE)>GL.

Right: <Application Prefix
(UPPERCASE)>GL_PST_TRANSACTION_NOT_POSTED

Wrong: TRANSACTION_NOT_POSTED

Use a Group Identifier as the Second Word in Each Message Name

Use a group identifier such as an acronym for the form or module that uses the message. If the message is used by more than one module, use ALL as the group identifier.



See Naming Standards for structure of form names. The DDDD component corresponds to the group identifier you should use here.

Message Writing Standards

Begin Messages with a Capital Letter

Capitalize the first character of a message. Do not capitalize every word.

Right: At last zone

Wrong: at last zone

Wrong: At Last Zone

Avoid Trailing Punctuation

Use punctuation only where needed for clarity. Commas are important, as are periods between sentences or sentence fragments.

Be Precise and Concise

Treat message text as formal written prose. Use proper sentence construction, punctuation and grammar. Use complete sentences. Be succinct and omit unnecessary words.

Right: You cannot add lines to a completed requisition

Wrong: You cannot append any more new lines to a requisition that has already been completed

Wrong: Requisition complete—cannot add lines

Avoid Ambiguous Words

Try to use words that have only one meaning. Avoid words with data processing connotations unless you are referring to a specific application function. Words found on a standard keyboard are likely to cause confusion.

Right: Choose another payment plan

Wrong: Select another payment plan

Right: Please save your work before continuing

Wrong: Please commit the transaction before continuing

Say “Please” Wherever Possible

Be polite. When a message contains instructions, and the message is short enough to display on the message line with room to space, include the word “please”.

Right: Please enter an amount greater than zero

Wrong: Enter an amount greater than zero

Use Terms Consistent with Form Boilerplate

Refer to a form field by its correct name. If a field is labeled “Sales Representative,” do not use the message “Please enter a different salesperson.”

Address the User as “You”

Talk to the user, not about the user. Users prefer friendly messages that address them directly and make them feel they control the application.

Right: You cannot delete this row

Wrong: The user cannot delete this row

Wrong: Cannot delete this row

Wrong: This row cannot be deleted

Prefer Solution-Oriented Messages

If there is a simple error that your user can easily correct (for example, a field without a value in it or a field containing an illegal value), then give directions for fixing the problem in your message. Do not describe the problem – tell how to fix it.

Right: Please enter a shipper

Wrong: Shipper is missing

Explain Why Your User Cannot Perform a Function

When a user tries to do something does not make sense in the current context, tell why it cannot be done.

Right: You cannot update this invoice because it is selected for payment

Wrong: You cannot update this invoice

Wrong: Invalid action

Right: You have already cleared this exception

Wrong: You cannot clear this exception

Use Active Voice

Avoid passive voice. If a message refers to a specific person (e.g., the user, the system administrator, another user), then the message should mention that person. Passive sentences are ambiguous regarding who or what is performing an action.

Right: You have canceled this process

Wrong: This process has been canceled

Wrong: This process has been canceled by you

If the message indicates that the application has performed some action, use the name of the custom application as the noun in the sentence.

Right: <Company Short Name> Purchasing cannot process your request

Wrong: Your request cannot be processed

Avoid Accusatory Messages

Do not insinuate that the user is at fault. Do not mention a user's mistake unless it pertains to the solution to the problem.

Right: The check number does not exist for this account. Please enter another number

Wrong: You entered an invalid check number

Use Imperative Voice

Sentences containing auxiliary verbs such as can, may, and should imply uncertainty. Use imperative voice to clearly communicate required actions.

Right: Enter a commission plan

Wrong: You can enter a commission plan
[or you can do something else...]

Naming Standards

The following standards should be followed for custom objects. In all examples, XXX represents the custom Application short name (i.e., <Application Prefix (UPPERCASE)>INV)



Attention: In the table below, literal text in the Name Format column is indicated in bold. Other components of the Name Format are described in the Explanation column

Object Type	Name Format	Explanation	Comments
Database Objects			
Table	XXX_DDDDDDD	DDDDDD = Description	Total length should not exceed 30 characters
View	XXX_DDDDDDD_V	DDDDDD = Description V = View	
Sequence	XXX_DDDDDDD_S	DDDDDD = Description S = Sequence	
Index	XXX_DDDDDDD_Tn	DDDDDD = Description T: U = Unique N = Non-Unique n = sequential number	Example: <Application Prefix (UPPERCASE)>PO_REGIONS_N2 is the second non-unique index on this custom table.
Stored Procedure	XXX_DDDDDDD	DDDDDD = Description (can be up to 32 char)	Follow same guidelines as table naming.
Procedure Package	XXX_DDDDDDD_PK	DDDDDD = Customization Abbreviation PK = Package (can be up to 32 char)	All procedures required for a single customization will be packaged together, so the description should reflect the customization as a whole.
Database Trigger	TABLE_XYZ_TRG	TABLE = Base table name X: B = Before A = After Y: I = Insert U = Update D = Delete Z: S = Statement R = Row TRG = Trigger	Example: PO_VENDORS_BUR_TRG is a Before Update trigger that fires on each Row. The base table name will need to be abbreviated if the total length exceeds 32 characters.
Operating System Files			
Form	XXXTDDDD.inp	T: D = Define U = Update I = Inquiry R = Report Launch DDDD = description	For files stored on MS-DOS servers or clients, the base filename cannot be longer than eight characters.
Report	XXXTDDDD.rex	T: R = Report U = Update DDDD = description	Although rare, some reports may primarily update tables.
SQL script	XXXTDDDD.sql	T: R = Report U = Update DDDD = description	
PL/SQL script	XXXTDDDD.pls	T: R = Report U = Update DDDD = description	
Procedure/Package install script	XXX_DDDDDDD_PK.sql	DDDDDD = Customization Abbreviation PK = Package	Same as above.
Database Trigger install script	TABLE_TRG.sql	TABLE = Base table name TRG = Trigger	Multiple trigger types will be combined into one install script
Database Object creation script	OBJECT.sql	OBJECT = name of database object	
Lookup table seed data script	XXXlkps.sql	lkps = Lookups	See example in Installation Scripts section below.
Grant/Synonym script	AAAgtBBB..sql	AAA = Application prefix where objects reside	

Object Type	Name Format	Explanation	Comments
		BBB = Application prefix of grantee	

Open and Closed Issues for this Deliverable

Open Issues

ID	Issue	Resolution	Responsibility	Target Date	Impact Date

Closed Issues

ID	Issue	Resolution	Responsibility	Target Date	Impact Date